

DSA - new 17

1. Queue $\rightarrow$ getMiddle $\Theta(1)$
 $\rightarrow$ popMiddle $\Theta(1)$, pop, push
 $\nrightarrow$ DS?
 $\rightarrow$ explain other op.

Queue op: - init(2)
- push(q, e)
- pop(q)
+ top(q) = front
- isEmpty(q)

DS: DLL with pointers to front, rear, middle, size.
choose middle as: $middle = \text{floor}((size+1)/2)$

DLLNode

info: TElem
next: $\uparrow$ DLLNode
prev: $\uparrow$ DLLNode

DLL

head: $\uparrow$ DLLNode
tail: $\uparrow$ DLLNode

Queue

fixed
cap.
(D.A.)

cap: Int
nElem: Int
elems: TElem[]
front: Int
rear: Int

$\nwarrow \nearrow$
on a Dyn. array
(fixed capacity)

(general)

DLLNode

info: TElem
next: $\uparrow$ DLLNode
prev: $\uparrow$ DLLNode

MiddleQueue

front: $\uparrow$ ~~Node~~ DLLNode
rear: $\uparrow$ DLLNode
middle: $\uparrow$ DLLNode
size: Int

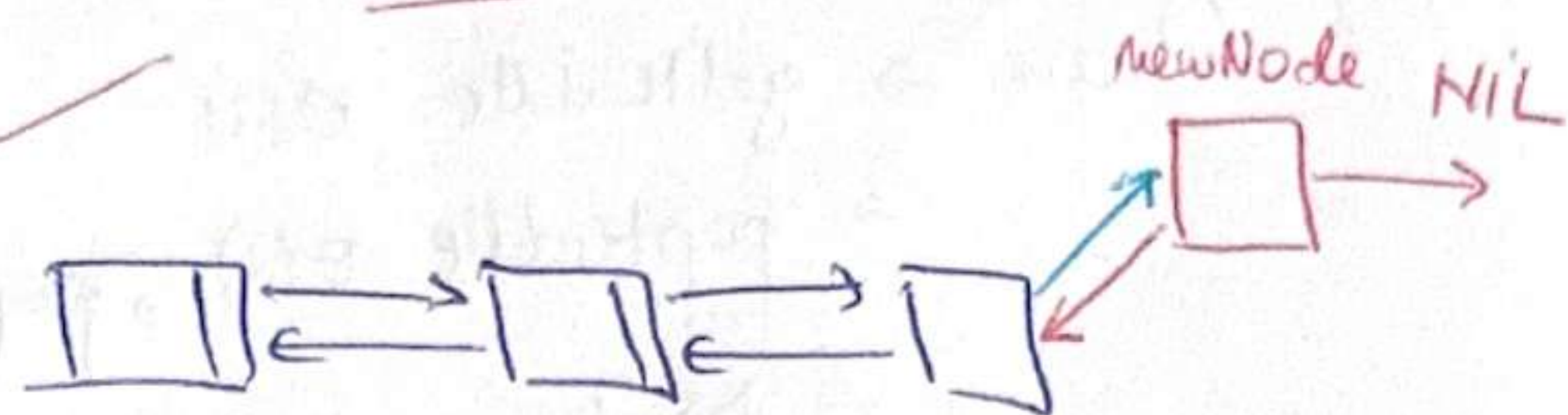
flexible cap. (DLL)

(our case)

getMiddle(q): return [q.middle].info
front(q): return [q.front].info
isEmpty(q): return q.size == 0
 $\left. \begin{array}{l} \text{getMiddle(q)} \\ \text{front(q)} \end{array} \right\} \Theta(1)$

subalgorithm push(g, e) is
 newNode ← allocate
 [newNode].info ← e
 [newNode].next ← NIL
 [newNode].prev ← g.rear

← push = add to rear



if g.size = 0 then
 g.front ← newNode
 g.rear ← newNode
 g.middle ← newNode

else

[g.rear].next ← newNode

g.rear ← newNode

if g.size mod 2 = 0 then

g.middle ← [g.middle].next

} reset the middle node after adding (size changed)

g.size ← g.size + 1

function pop(g) is

if g.size = 0 then
 @throw...

removed ← g.front

pop ← [removed].info

if g.size = 1 then

g.front ← NIL

g.rear ← NIL

g.middle ← NIL

else

if g.size mod 2 = 0 then

g.middle ← [g.middle].next

g.front ← [g.front].next

[g.front].prev ← NIL

g.size ← g.size - 1

function popMiddle (g) is

if g.size = 0 then
 @throw ...

removed ← g.middle

popMiddle ← [removed].info

if g.size = 1 then

 g.front ← Nil

 g.rear ← Nil

 g.middle ← Nil

else

 if g.size mod 2 = 1 then

 g.middle ← [removed].prev

 else

 g.middle ← [removed].next

 if [removed].prev ≠ Nil then

[[removed].prev].next ← [removed].next

 else

 g.front ← [removed].next

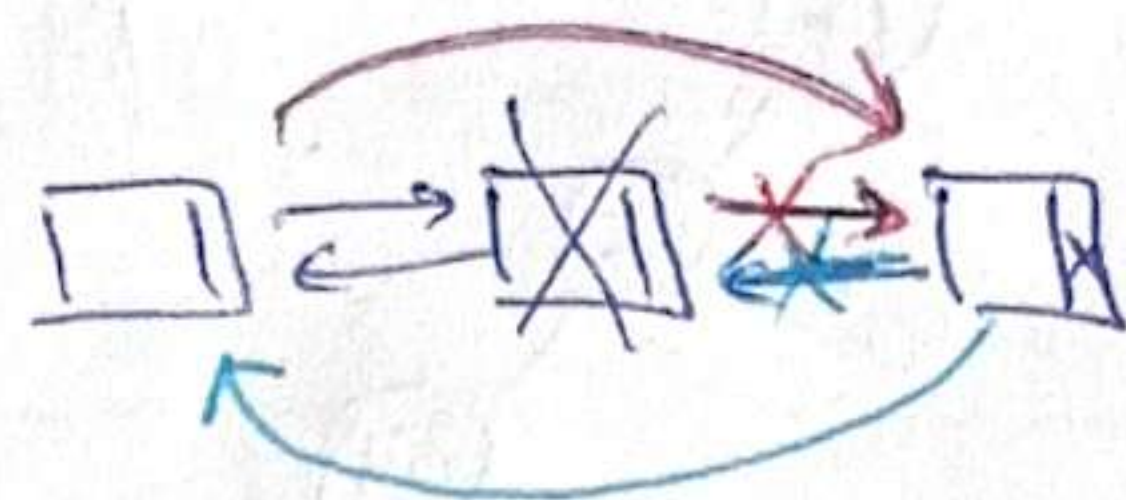
 if [removed].next ≠ Nil then

[[removed].next].prev ← [removed].prev

 else

 g.rear ← [removed].prev

g.size ← g.size - 1

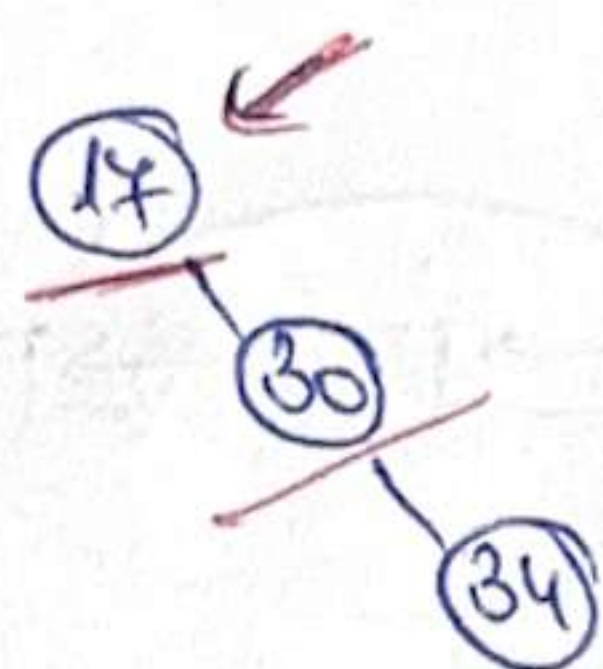


$O(1)$

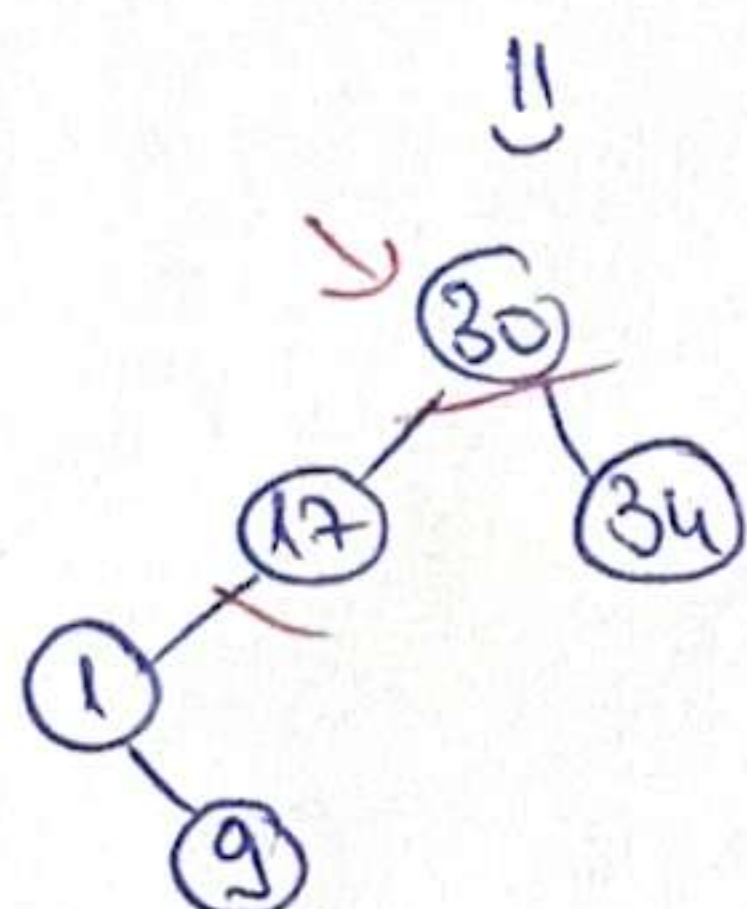
2. a) AVL: - insert: 17, 30, 34, 1, 9, 20, 25, 4, 35, 22, 31, 40.

- rotate

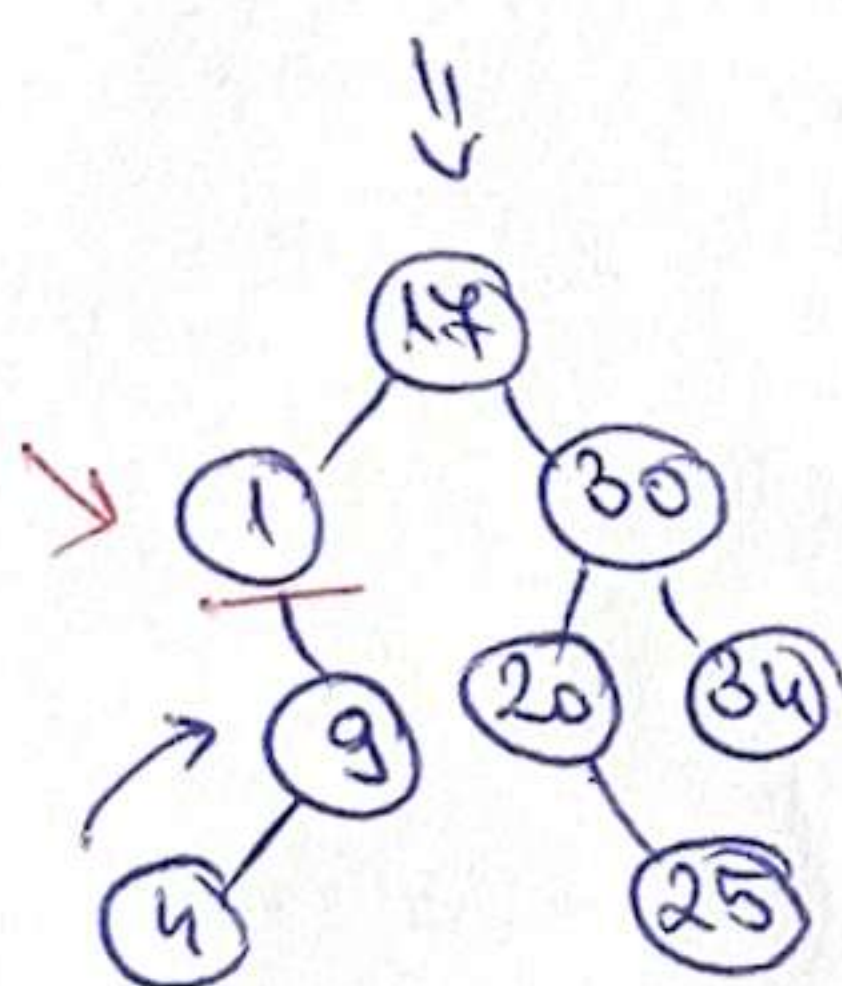
- remove 20 and replace with sth. from the left subtree.



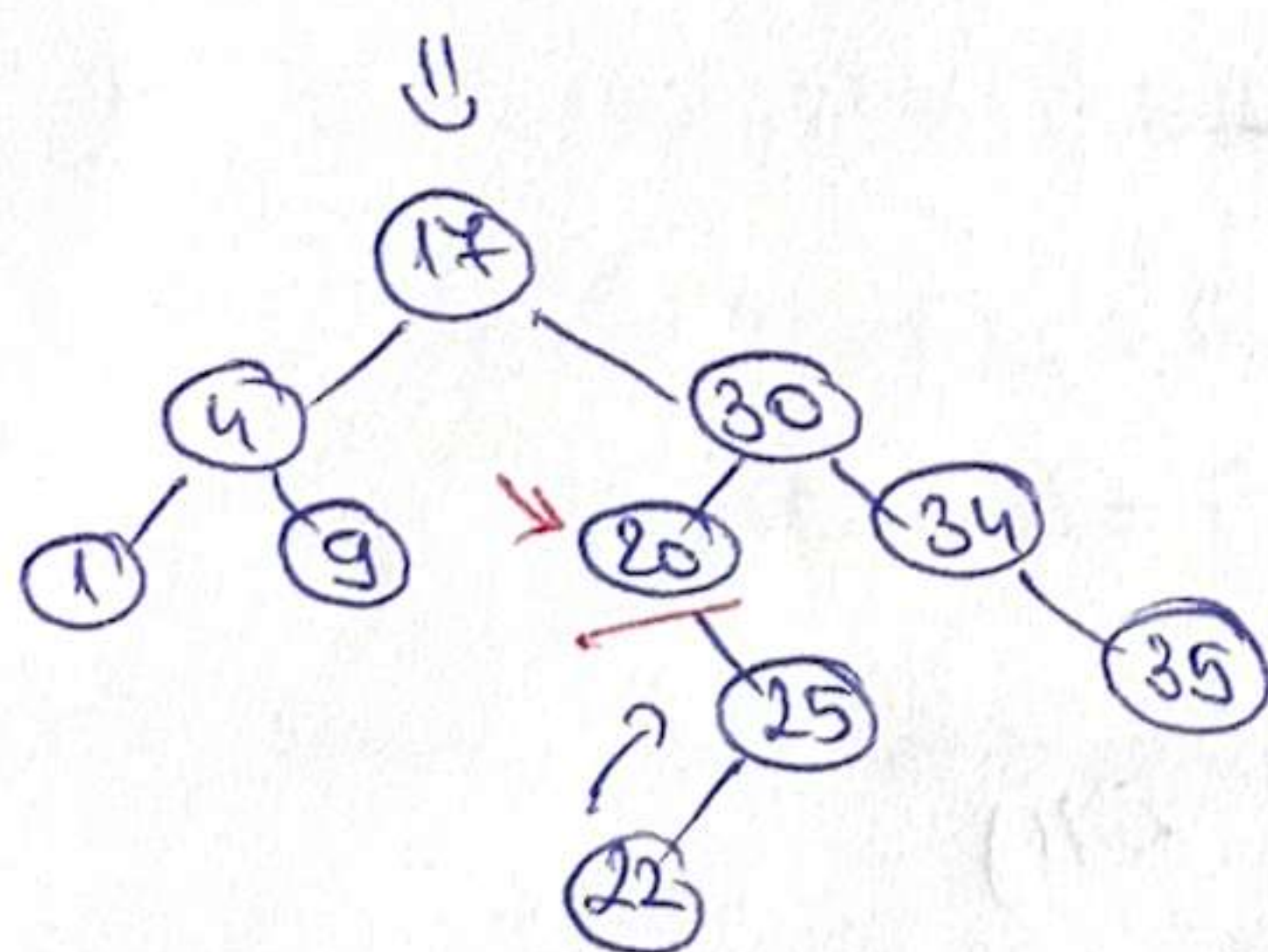
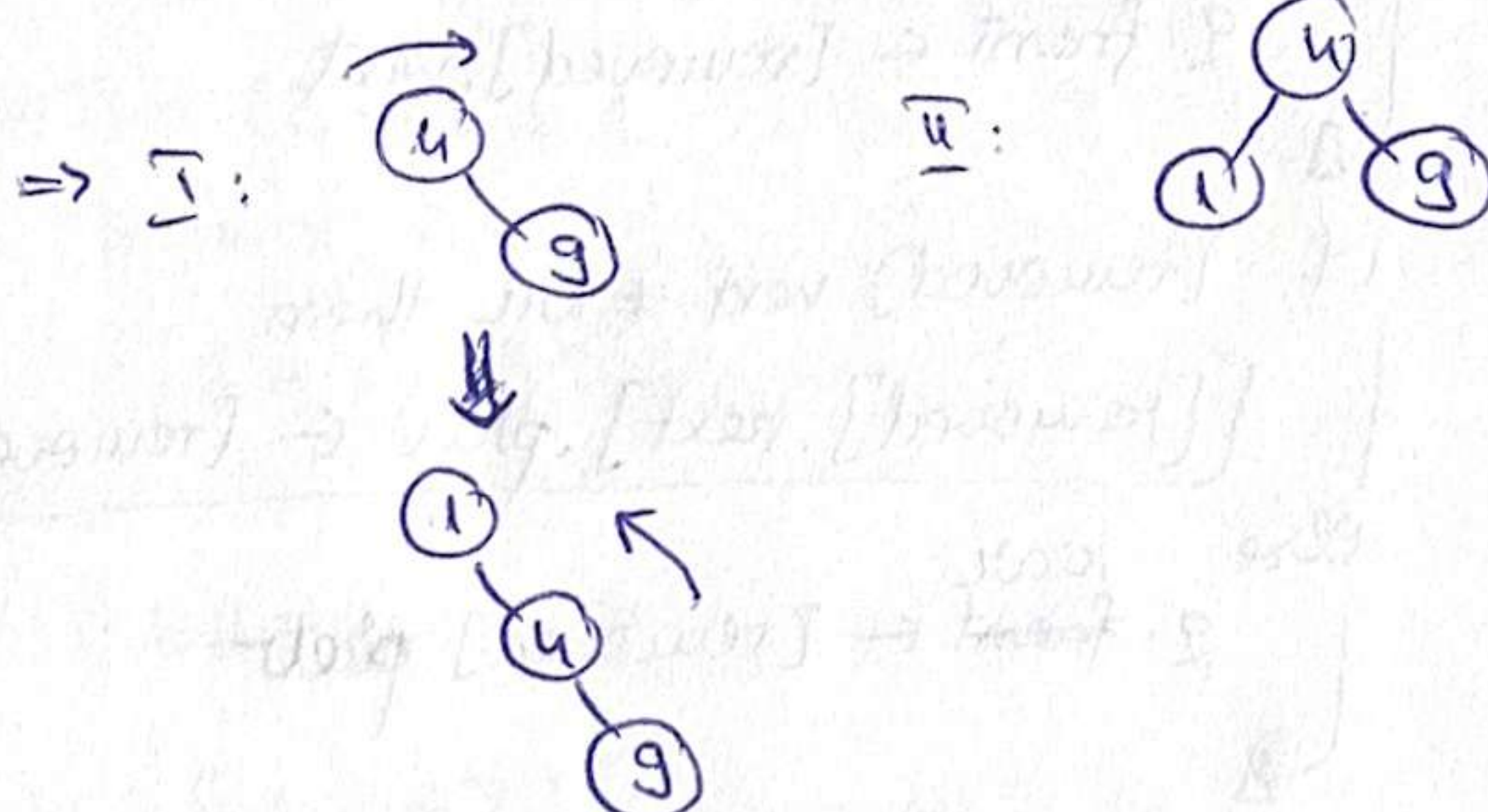
17 is HR, its ⁽³⁰⁾R child is HR $\Rightarrow$ RR $\Rightarrow$ SRL



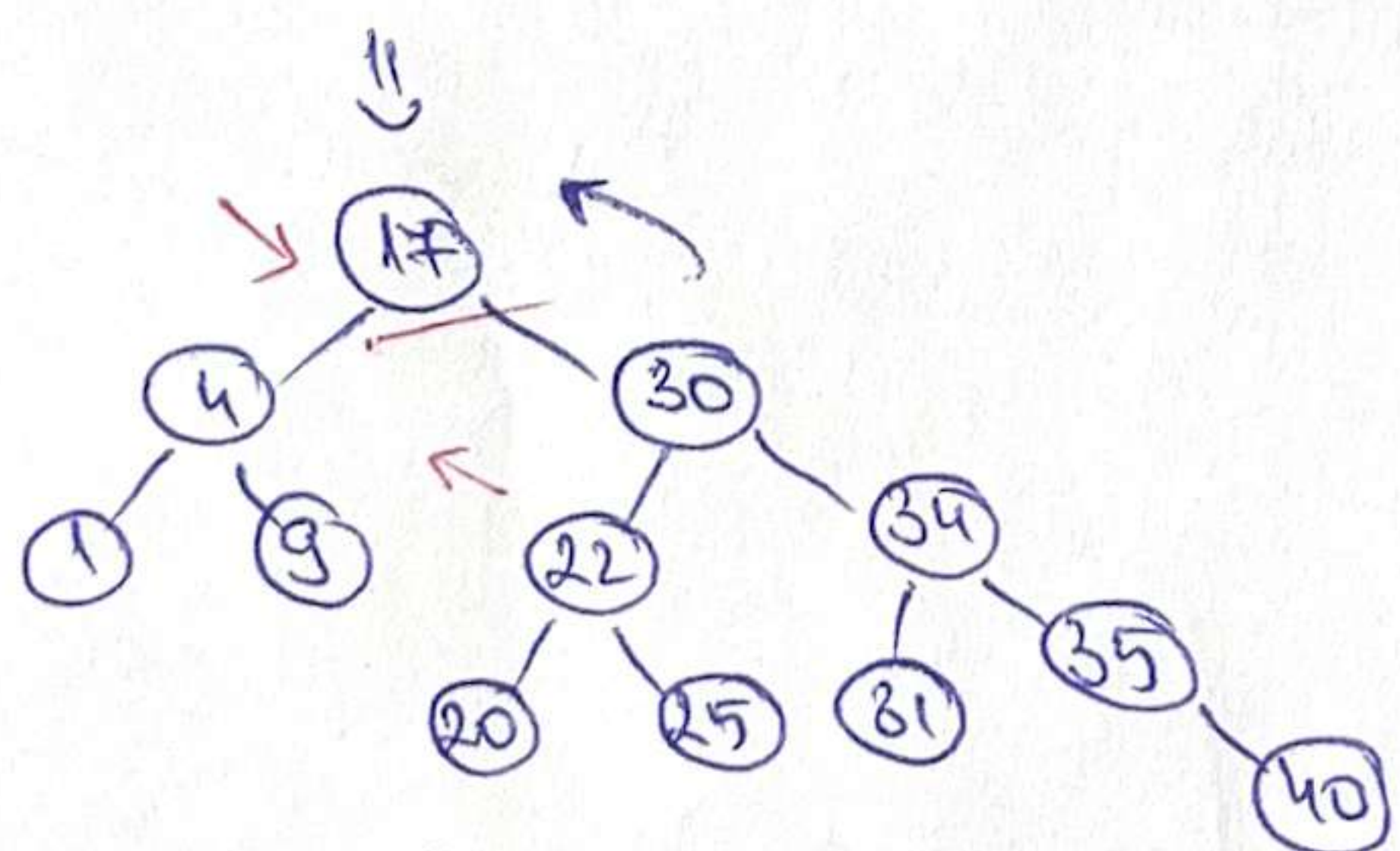
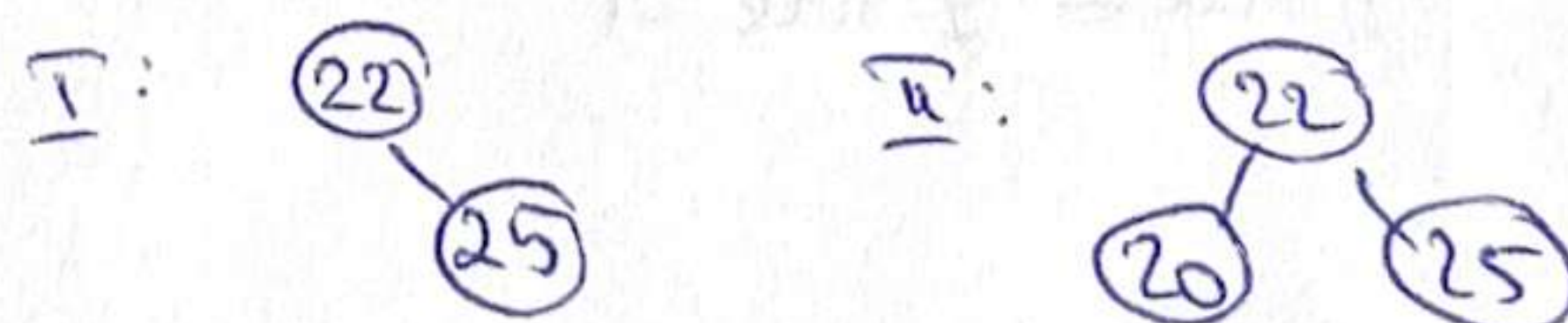
30 is HL, its L child (17) is HL $\Rightarrow$ LL $\Rightarrow$ SRR



1 is HR, its R child (9) is HL $\Rightarrow$ RL $\Rightarrow$ DRL

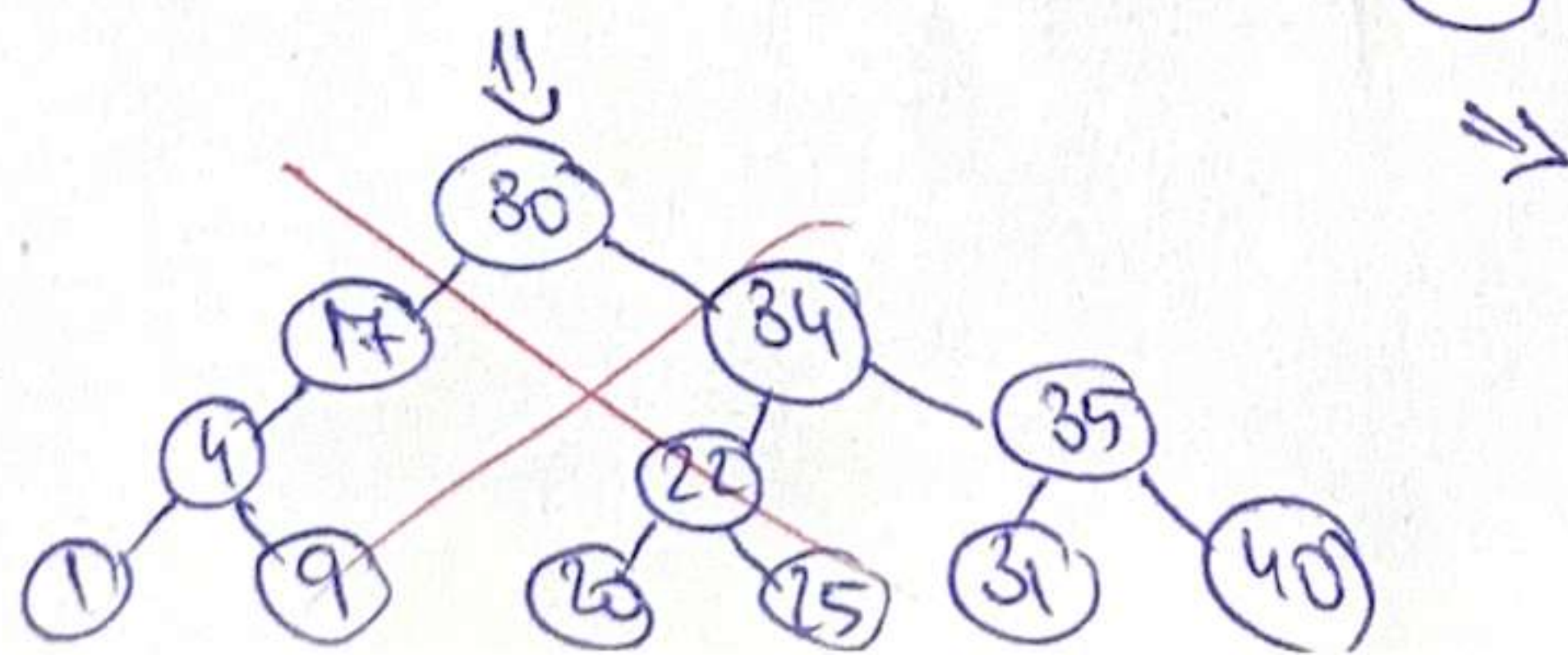
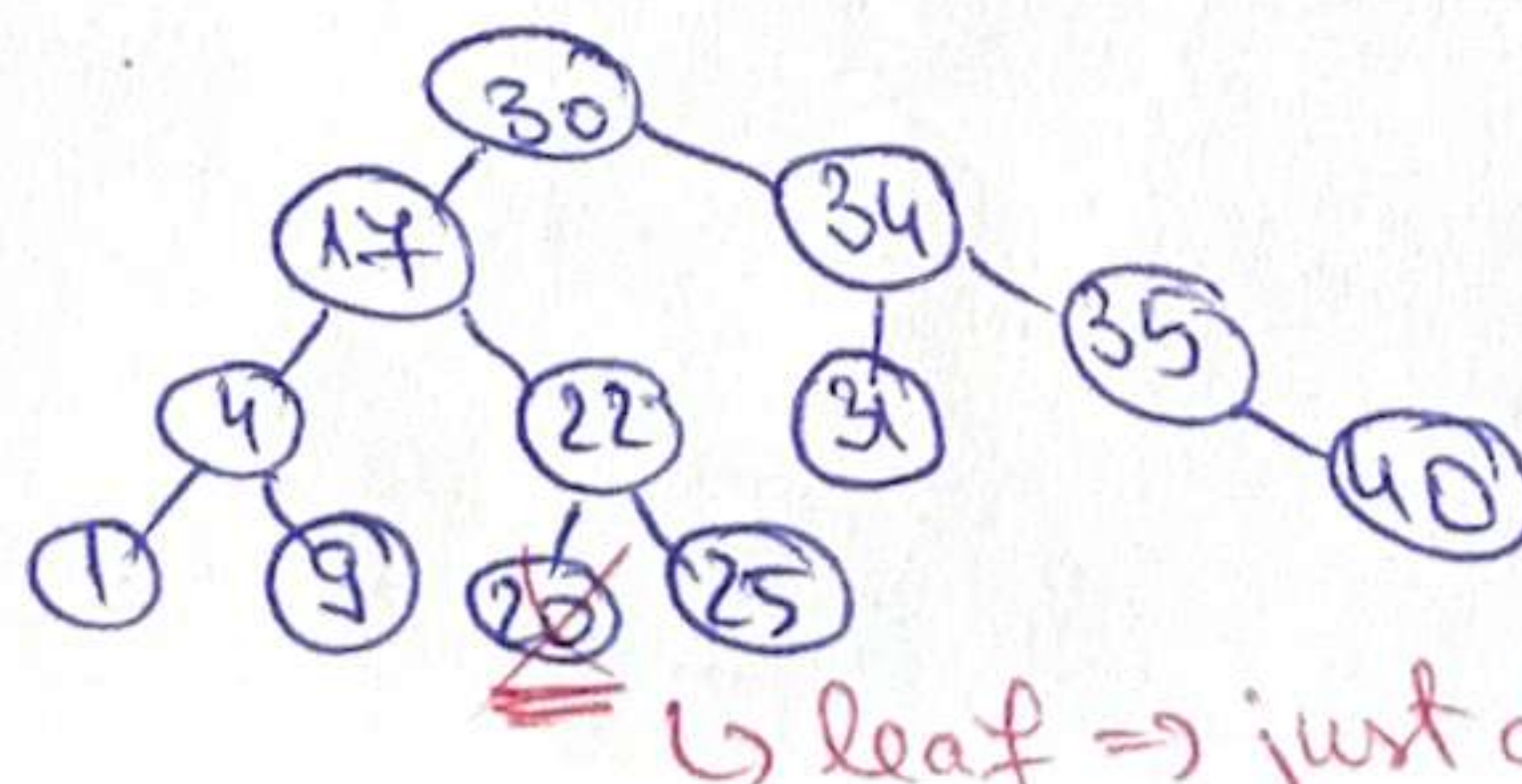


20 is HR, its R child (25) is HL $\Rightarrow$ RL $\Rightarrow$ DRL



17 is HR, its R child (30) is HR

$\Rightarrow$ RR $\Rightarrow$ SRL



$\hookrightarrow$ leaf $\Rightarrow$ just delete it.

- b) $m=13$, HT, quadratic probing with $c_1=1$, $c_2=2$, div. method.
insert: 30, 19, 42, 50, 16, 136, 32.

Quadratic probing $\leftarrow$ OPEN ADDRESSING (probe sequence)

$$h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \% m = (h'(k) + i + 2i^2) \% 13.$$

$$h'(30) = 30 \% 13 = 4$$

$$h'(19) = 6$$

$$h'(42) = 3$$

$$h'(50) = 11$$

$$h'(16) = 3$$

$$h'(136) = 6$$

$$h'(32) = 6$$

0	1	2	3	4	5	6	7	8	9	10	11	12
16	32		42	30		19			136		50	

$$h(30, 0) = (4 + 0) \% 13 = 4 \checkmark$$

$$h(19, 0) = (6) \% 13 = 6 \checkmark$$

$$h(42, 0) = (3) \% 13 = 3 \checkmark$$

$$h(50, 0) = 11 \% 13 = 11 \checkmark$$

$$h(16, 0) = 3 \% 13 = 3 \times$$

$$\hookrightarrow h(16, 1) = (3 + 1 + 2) \% 13 = 6 \% 13 = 6 \times$$

$$\hookrightarrow h(16, 2) = (3 + 2 + 8) \% 13 = 13 \% 13 = 0 \checkmark$$

$$h(136, 0) = 6 \% 13 = 6 \times$$

$$\hookrightarrow h(136, 1) = (6 + 1 + 2) \% 13 = 9 \checkmark$$

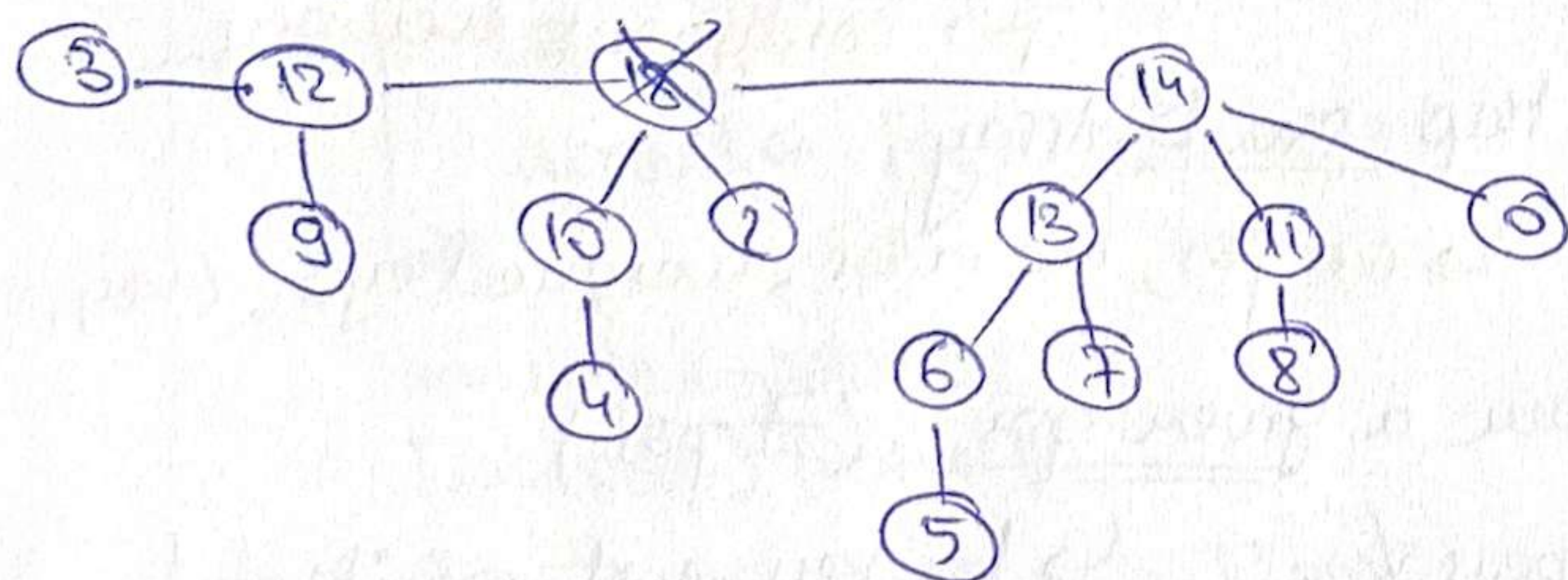
$$h(32, 0) = 6 \% 13 \times$$

$$\hookrightarrow h(32, 1) = (6 + 1 + 2) \% 13 = 9 \times$$

$$\hookrightarrow h(32, 2) = (6 + 2 + 8) \% 13 = 16 \% 13 = 3$$

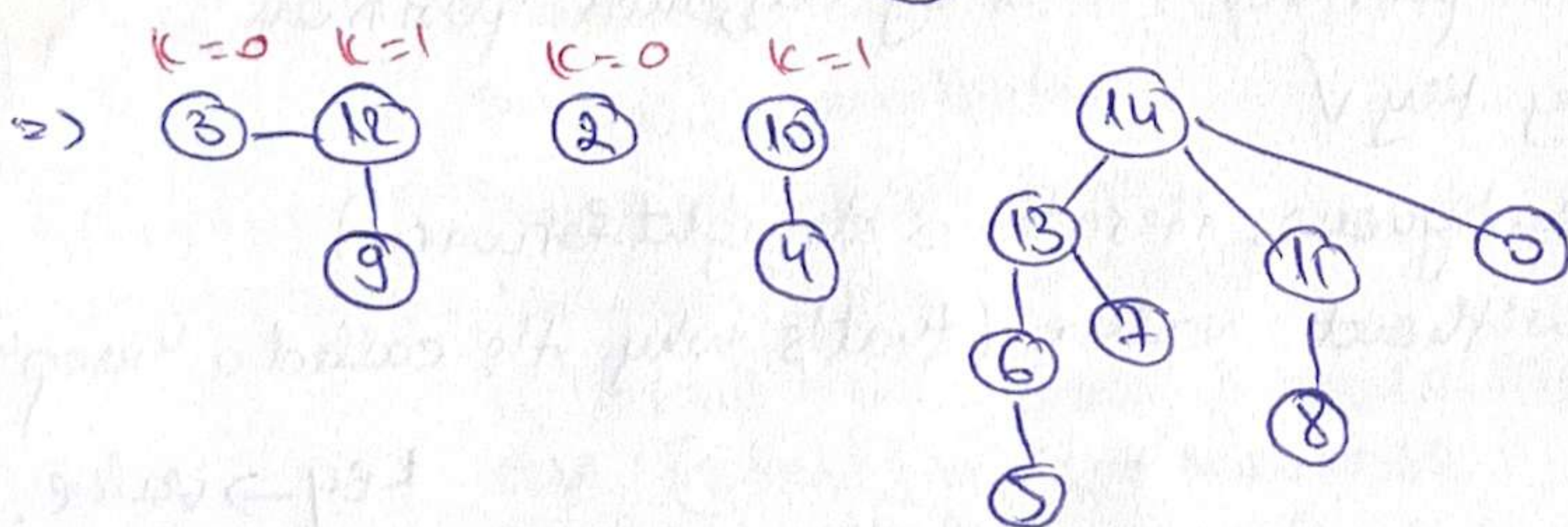
$$\hookrightarrow h(32, 3) = (6 + 3 + 18) \% 13 = 27 \% 13 = 1 \checkmark$$

c) Priority Queue on binomial heap.



max priority heap.

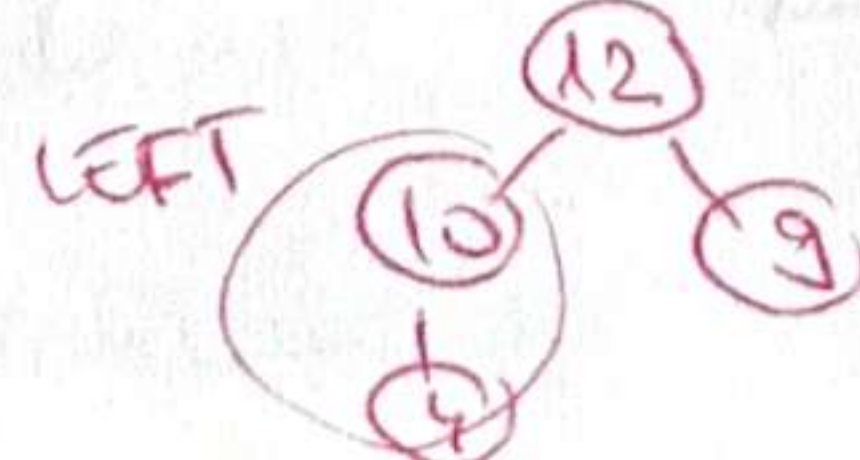
$\downarrow$
pop LARGEST $\Rightarrow$ (18).

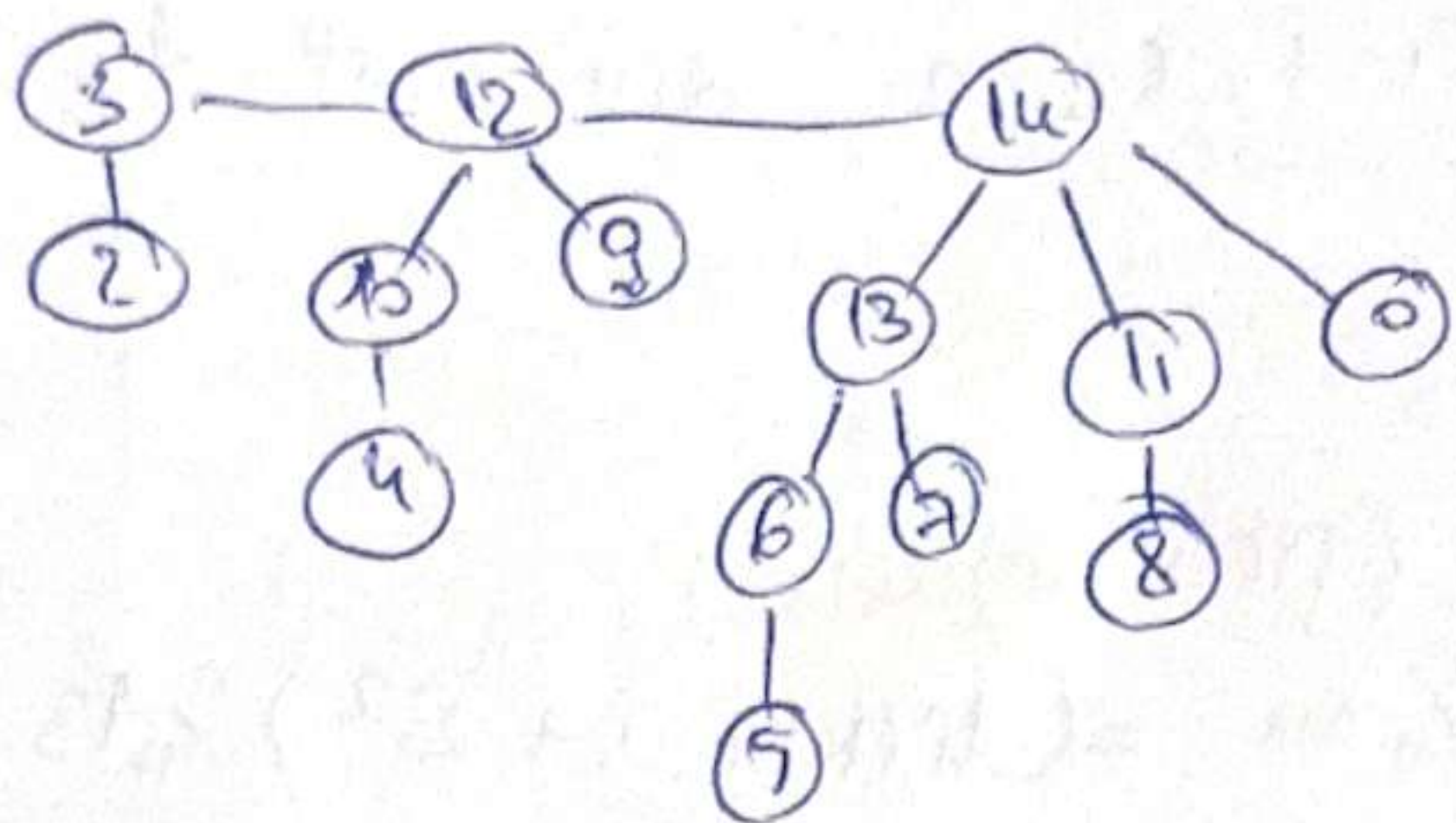


$k=0 \Rightarrow$ merge $\Rightarrow$



$k=1 \Rightarrow$ merge $\Rightarrow$





3.

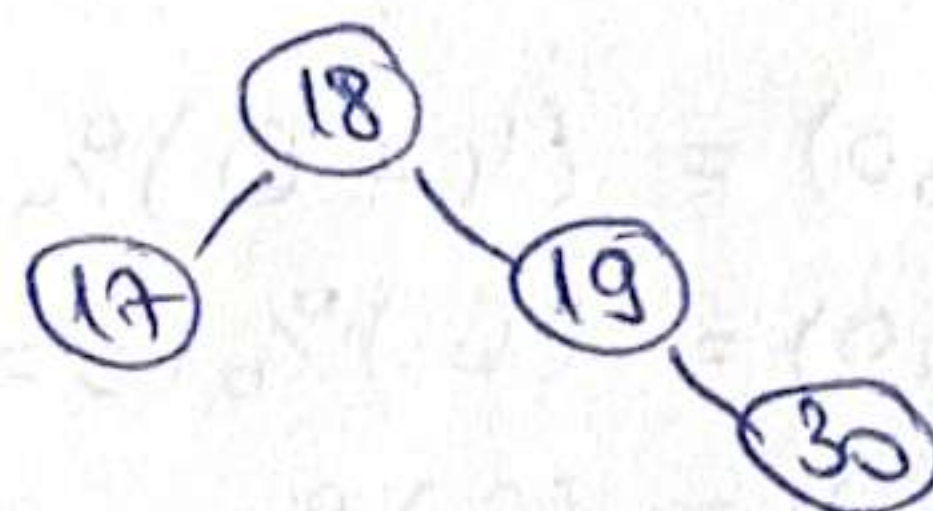
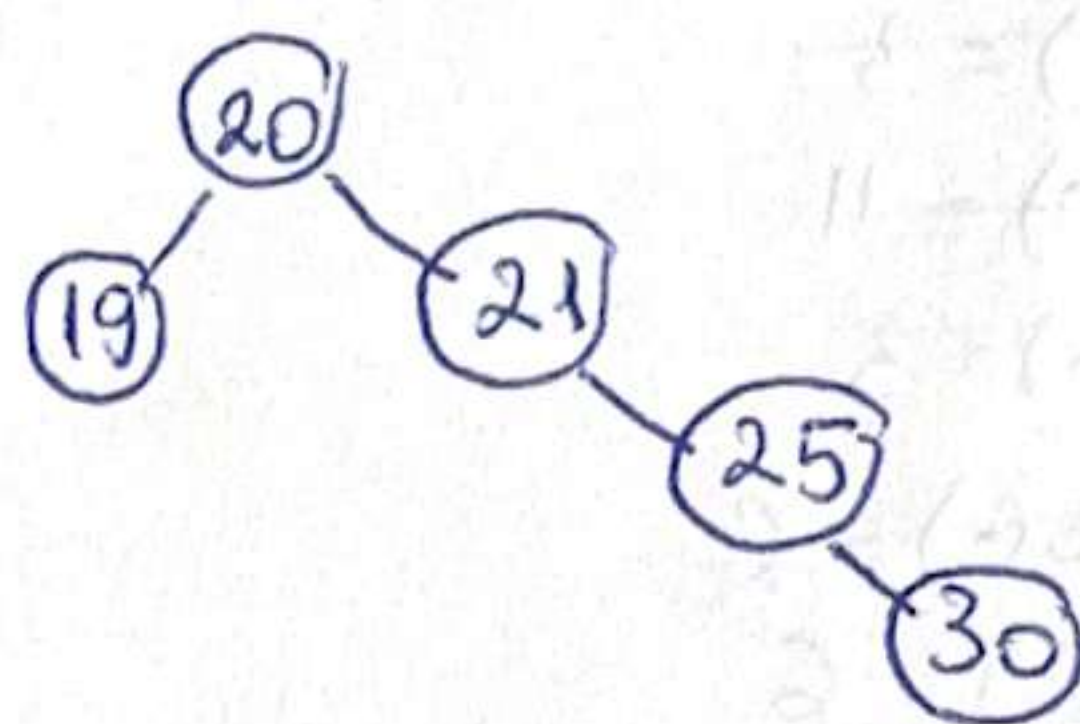
1) BST. $\rightarrow$ 19, 30.

inorder: Left Root Right

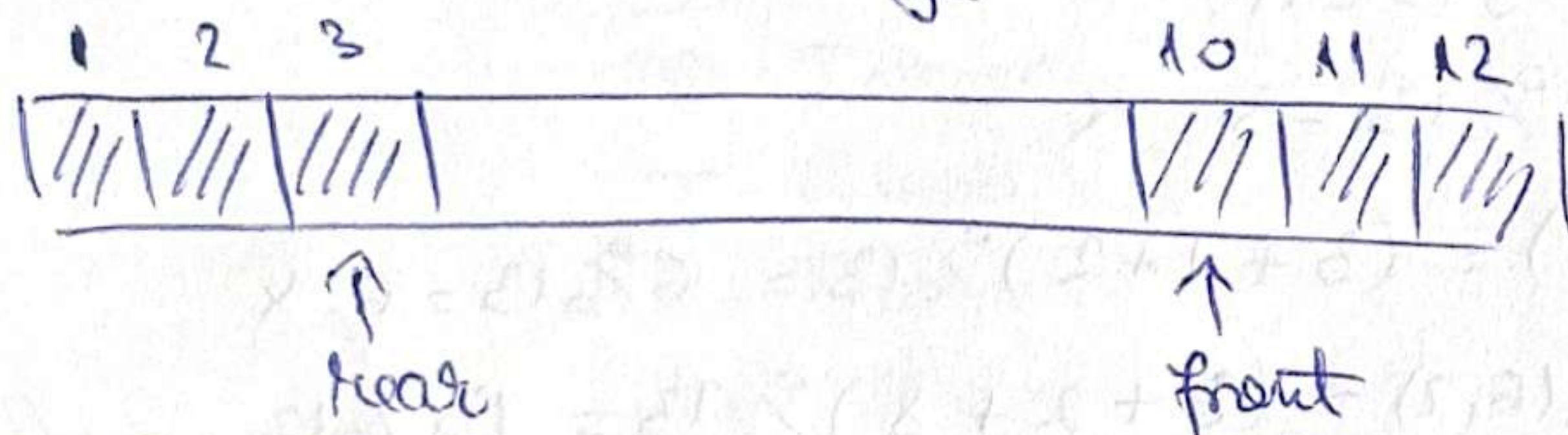
$\Rightarrow$ 19 30 25 21

$\Rightarrow$ 19 20 21 25 30

$\Rightarrow$ 17 18 19 30 a) 19



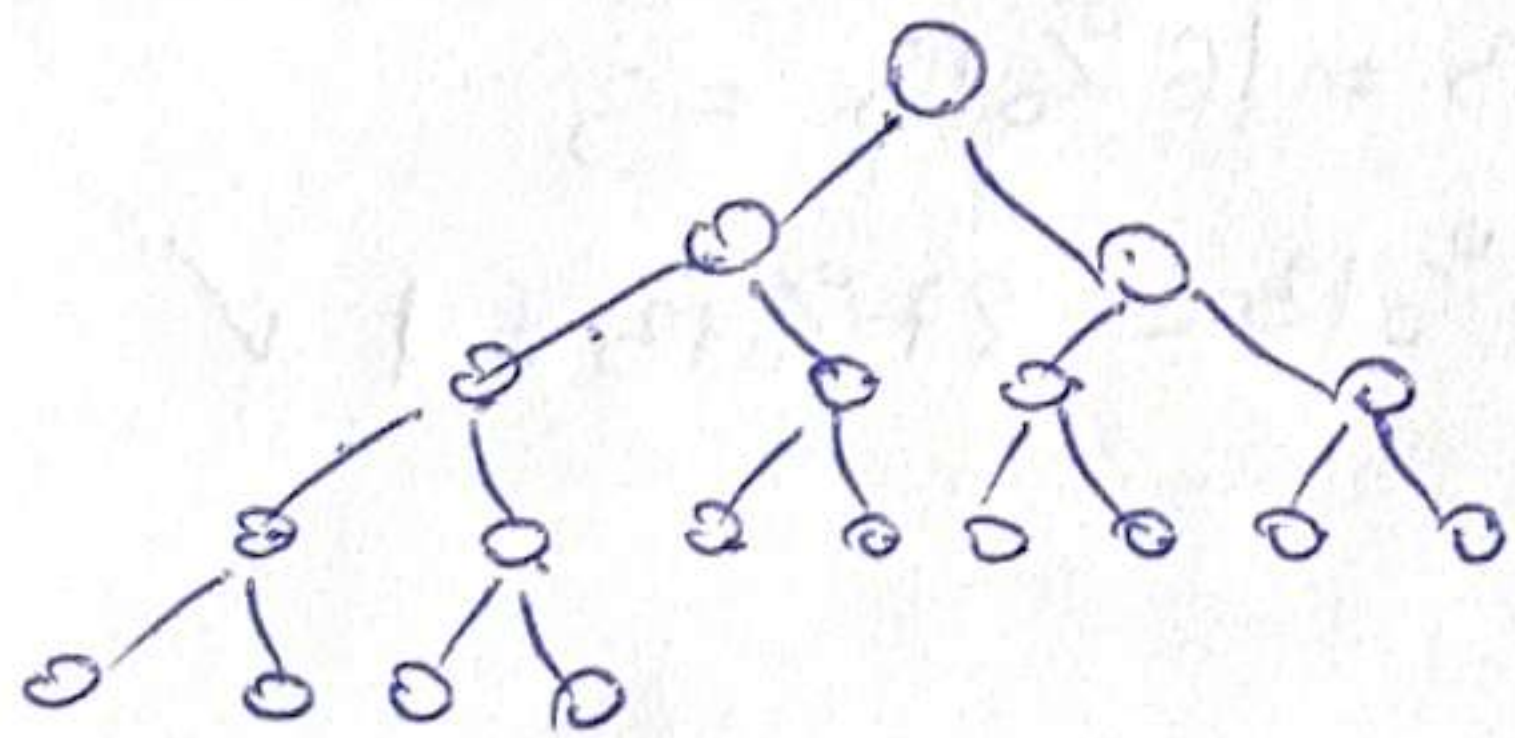
2) Queue on circular array, 12 pos.



d) 6 elems in the queue

3) $\alpha = \frac{m}{m}$ b)

4) Min depth, 19 nodes. $\Rightarrow$ c) 4



$\rightarrow$ contiguous memory

5) $\nexists$ this op. on a Map on D. Array.

$\hookrightarrow$ no pos, no order, unique keys, (key, value)

$\nexists$ b) remove a pair from a given pos. ($\nexists$ pos.)

a. insert (key, value) pair $\checkmark$

c. search value by key $\checkmark$

d. iterator $\checkmark$ (contiguous, iterator is straightforward).

$\nexists$ e) insert key without value (that's why it's called a "map").

Key $\rightarrow$ value. 6

4. Set $\rightarrow$ on a Linked HT.

- $\rightarrow$ no pos, no order
- $\rightarrow$ unique; $\nexists$ remove from pos.
- $\rightarrow$ repres. of a Set
- $\rightarrow$ add op.
- $\rightarrow$ complexity

$\rightarrow$ HT with separate chaining
 $\rightarrow$ iterator displays elems. in the order of insertion!

Linked HT

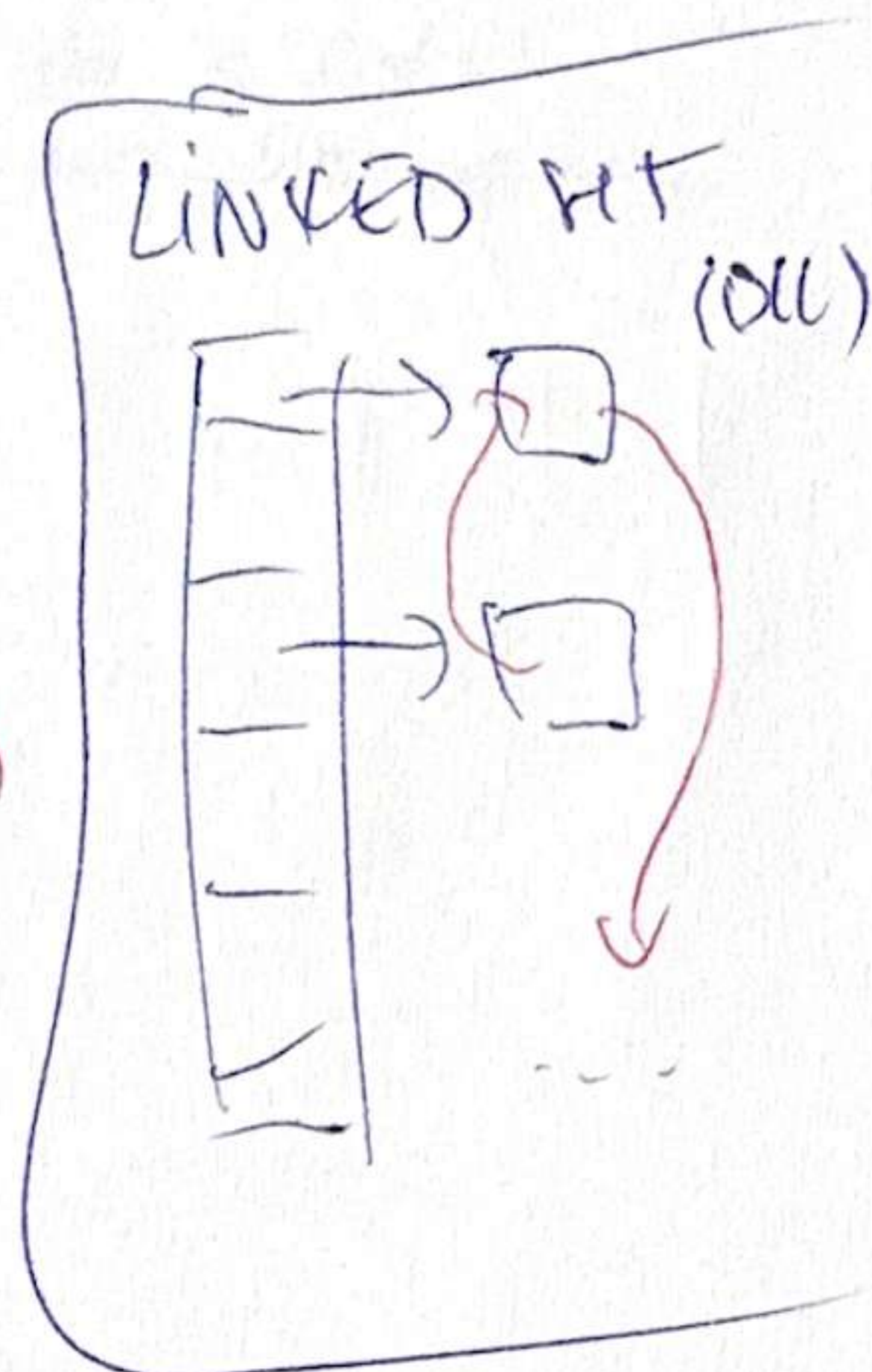
Node

info: Tkey
 nextH: $\uparrow$ Node
 nextL: $\uparrow$ Node
 prevL: $\uparrow$ Node

Linked HT

m: Integer
 T: ($\uparrow$ Node)[$\uparrow$]
 h: Tfunction
 tail: $\uparrow$ Node
 head: $\uparrow$ Node

(general)



Node

info: TElem
 nextH: $\uparrow$ Node
 nextL: $\uparrow$ Node
 prevL: $\uparrow$ Node

order of insertion

Set

m: Integer
 T: ($\uparrow$ Node)[$\uparrow$]
 h: Tfunction
 tail: $\uparrow$ Node
 head: $\uparrow$ Node

(our case)

characteristic to a set.

(cap, mElem, elems from DA)

mElem: Integer

insert there (insertion order)

subalgorithm add(s, e) is

$\rightarrow$ at tail of the insertion order list

pos $\leftarrow$ s.h(e)
 current $\leftarrow$ s.T[pos]
 while current $\neq$ Nil execute
 if [current].info = e then
 @ throw ...
 current $\leftarrow$ [current].nextH

newNode $\leftarrow$ allocate
 [newNode].info $\leftarrow$ e

BC: $\Theta(1)$
 WC: $\Theta(m)$
 T: $O(m)$

insert in HT

[newNode].nextH $\leftarrow$ s.T[pos]
 s.T[pos] $\leftarrow$ newNode

insert at the end of the insertion order list

[newNode].nextL $\leftarrow$ Nil
 [newNode].prevL $\leftarrow$ s.tail
 if s.tail = Nil then s.head $\leftarrow$ newNode
 else [s.tail].nextL $\leftarrow$ newNode
 s.tail $\leftarrow$ newNode

erg
nd
ep
ue
ll
iv
n
g
l